

UNIVERSITÀ
DEGLI STUDI
DI TORINO

Progetto parte 1
CKY Elfico

Frattarola Marco, Ghione Alessio

2024-2025

Indice

1 Introduzione **3**
1.1 Parsificare il linguaggio degli Eldar 3

2 L'Algoritmo CKY **3**
2.1 Struttura e Funzionamento 3
2.2 Gestione delle Regole Unarie 3

3 Pipeline **4**
3.1 Implementazione in Python 4
3.2 Definizione della Grammatica Quenya 4
3.3 Semantica (Bonus Track) 5

4 Analisi dei Risultati **5**
4.1 Test sulla grammatica L1 (Inglese) 5
4.2 Parsing Sintattico Quenya 6

4.3 Parsing Semantico 6

1 Introduzione

1.1 Parsificare il linguaggio degli Eldar

Come progetto ci siamo focalizzati sull'implementazione dell'algoritmo di parsing CKY (Cocke-Younger-Kasami), un algoritmo di programmazione dinamica bottom-up utilizzato per l'analisi sintattica di grammatiche libere dal contesto (CFG).

L'obiettivo principale del laboratorio è stato duplice: da un lato validare l'algoritmo implementato *from scratch* su una grammatica inglese standard (la L1 di Jurafsky), dall'altro applicare tali tecnologie a una lingua artificiale complessa: il **Quenya**, l'Elfico ideato da J.R.R. Tolkien.

Abbiamo inoltre esteso l'analisi introducendo una componente semantica (Bonus Track) per tradurre le frasi elfiche in rappresentazioni logiche basate sul calcolo dei predicati (First Order Logic).

2 L'Algoritmo CKY

2.1 Struttura e Funzionamento

Per calcolare l'albero sintattico delle frasi, abbiamo implementato la classe `CKYParser`. L'algoritmo richiede che la grammatica sia in **Forma Normale di Chomsky** (CNF), ovvero che le regole di produzione siano esclusivamente della forma:

- $A \to B\ C$ (regole binarie con due non-terminali)
- $A \to w$ (regole lessicali con un terminale)

L'algoritmo riempie una tabella triangolare di dimensione $(n+1) \times (n+1)$, dove n è il numero di parole della frase. Ogni cella $[i][j]$ contiene l'insieme dei non-terminali che possono generare la sottostringa che va dall'indice i all'indice j.

2.2 Gestione delle Regole Unarie

Sebbene il CKY standard operi strettamente su grammatiche CNF binarie, abbiamo deciso di implementare un metodo `add_unary_closure` all'interno della pipeline. Questo permette di gestire catene di regole unarie (es. $NP \to Nominal \to Noun$) senza dover esplodere eccessivamente la grammatica, rendendo la scrittura delle regole più naturale pur mantenendo l'efficienza polinomiale $O(n^3)$.

3 Pipeline

3.1 Implementazione in Python

Il nucleo del progetto risiede nel file `cky_parser.py`. La struttura dati principale è definita nella classe `Grammar`, che separa le regole sintattiche dal lessico.

```
# Ciclo principale sui possibili punti di split k
for i in range(j - 2, -1, -1):
    for k in range(i + 1, j):
        for B in self.table[i][k]:
            for C in self.table[k][j]:
                # Cerca regole della forma A -> B C
                lhs_list = self.grammar.get_rules_by_body((B, C))

                for A in lhs_list:
                    self.table[i][j].add(A)
                    # Salvataggio back-pointer
                    self.back_pointers[i][j][A].append(('binary', k, B, C))
```

Listing 1: Snippet del metodo di parsing CKY

Il parser non restituisce solo un valore booleano di accettazione, ma ricostruisce l'intero albero sintattico (o gli alberi, in caso di ambiguità) navigando ricorsivamente i `back_pointers` salvati durante la fase di riempimento della tabella.

3.2 Definizione della Grammatica Quenya

Nel file `quenya_grammar.py` abbiamo definito le regole sintattiche per parsificare le frasi assegnate. Abbiamo convertito le strutture grammaticali del Quenya in una CNF compatibile con il nostro parser.

Le frasi analizzate sono:

- *I hesto samë macil* (Il capitano ha una spada)
- *I aran tíra aiwi* (Il re guarda gli uccelli)
- *Nanye lumba* (Non sono stanco)
- *Nálmë eldar* (Siamo elfi)
- *I atan antanë i eldan tecil* (L'uomo diede all'elfo una penna)

Per gestire verbi ditransitivi come "antanë" (diede), abbiamo strutturato il Sintagma Verbale (VP) per accettare sequenze multiple di Sintagmi Nominali (NP), binarizzando le regole come segue:

$$VP \rightarrow Verb\ NP\text{-}NP$$

$$NP\text{-}NP \rightarrow NP\ NP$$

3.3 Semantica (Bonus Track)

Per la parte facoltativa, abbiamo utilizzato `nltk.grammar.FeatureGrammar` nel file `quenya_semantic.py`. Questo ci ha permesso di associare a ogni regola sintattica una λ-espressione semantica.

Esempio di definizione semantica per il verbo transitivo "samë" (avere):

```
TV[NUM=sg,SEM=<\X x.X(\y.have(x,y))>] -> 'same'
```

Questo approccio permette di comporre il significato delle frasi calcolando l'applicazione delle funzioni lambda nodo per nodo, risalendo l'albero sintattico.

4 Analisi dei Risultati

4.1 Test sulla grammatica L1 (Inglese)

Inizialmente, il parser è stato testato sulla grammatica di Jurafsky. Il parsing della frase *"Does she prefer a morning flight"* ha prodotto correttamente l'albero sintattico atteso, confermando la correttezza della gestione dei cicli interni e dei puntatori per la ricostruzione.

```
[S
  [Aux-NP
    [Aux does]
    [NP
      [Pronoun she]
    ]
  ]
  [VP
    [Verb prefer]
    [NP
      [Det a]
      [Nominal
        [Nominal
          [Noun morning]
        ]
        [Noun flight]
      ]
    ]
  ]
]
```

Figura 1: Output testuale dell'albero sintattico per la L1

4.2 Parsing Sintattico Quenya

L'applicazione del parser alle frasi in Quenya ha dato esito positivo per tutte e cinque le frasi target. Di seguito riportiamo l'albero generato per la frase complessa ditransitiva: *"I atan antanë i eldan tecil"*.

```
Parsing:   I atan antanë i eldan tecil
Parsing riuscito.

--- Parse Tree 1 ---
[S
  [NP
    [Det i]
    [Noun atan]
  ]
  [VP
    [Verb antanë]
    [NP-NP
      [NP
        [Det i]
        [Noun eldan]
      ]
      [NP
        [Noun tecil]
      ]
    ]
  ]
]
```

Nota: Nella conversione in CNF, alcune etichette come PP sono state riadattate per accomodare la struttura ternaria del verbo ditransitivo.

4.3 Parsing Semantico

Utilizzando la Feature Grammar di NLTK, abbiamo ottenuto le rappresentazioni logiche (First Order Logic) delle frasi. L'analisi semantica risolve correttamente le dipendenze tra soggetto, oggetto e predicato.

Per la frase *"I hesto samë macil"* (Il capitano ha una spada), il sistema ha prodotto:

- **Albero:** (S (NP (Det I) (Nom (N hesto))) (VP (TV samë) (NP (Nom (N macil)))))
- **Semantica:** $\exists x.(hesto(x) \wedge \exists z.(macil(z) \wedge have(x, z)))$

Ciò dimostra che il sistema ha correttamente identificato "hesto" come entità x, "macil" come entità z e le ha legate tramite la relazione $have(x, z)$.